

RecruitX AI — Bug Analysis & Fix Plan

Based on recorded clip review (June 30, 2026). Covers all four critical issues impacting conversational flow, ASR accuracy, contextual memory, and loop recovery.

Table of Contents

Codebase Architecture	2
Issue 1 — VAD / Turn-Taking & Interruption Logic	2
What the code actually does today	2
Bug A — "Incomplete sentence" detection too narrow	2
Bug B — Barge-in is detection-only, not audio cancellation	2
What to implement	3
Before / After — VAD	3
Issue 2 — ASR Phonetic Mishearings	3
What the code actually does today	3
What to implement	4
Before / After — ASR	5
Issue 3 — Contextual Memory & Gaslighting	6
What the code actually does today	6
What to implement	6
Before / After — Memory & Gaslighting	7
Issue 4 — Repetitive Loop Fallbacks	8
What the code actually does today	8
What to implement	8
Before / After — Loop Fallbacks	9
What's In Code vs What's Missing	9
Priority Order for Implementation	10

Codebase Architecture

File	Role
pipeline.py	VAD timers, silence detection, LLM orchestration, TTS pipeline, interview state
recall_client.py	Deepgram ASR config, Recall.ai bot management
main.py	FastAPI server, webhook receiver, session management

Issue 1 — VAD / Turn-Taking & Interruption Logic

What the code actually does today

Silence thresholds (pipeline.py:17-26)

```
SILENCE_SHORT      = 0.8    # 1-5 words
SILENCE_MEDIUM     = 1.2    # 6-15 words
SILENCE_LONG       = 2.0    # 16-35 words
SILENCE_XLONG      = 3.5    # 35+ words
SILENCE_INCOMPLETE = 1.8    # ends with trailing word like "and", "because"
SILENCE_INTERRUPTED = 0.5   # candidate spoke over bot
```

The `_adaptive_timeout()` at pipeline.py:318 scales by word count and adds a 0.5s buffer for very short fragments (≤ 4 words). However there are two concrete bugs:

Bug A — "Incomplete sentence" detection too narrow

The `_TRAILING_WORDS` set at pipeline.py:35 was intentionally trimmed to avoid false-positive 1.8s waits on Indian English sentence-enders. But it no longer catches patterns like:

- "so what I did was..." — ends with "was" → NOT in the set, treated as complete
- "I used React for the..." — ends with "the" ✓ caught, but only 0.8s timer fires

With Deepgram endpointing=300ms, a candidate mid-sentence gets 300ms of silence, Deepgram fires a segment, the pipeline starts a 1.2s timer — total ~1.5s before the bot can interrupt. That is tight for a technical explanation.

Bug B — Barge-in is detection-only, not audio cancellation

In `on_transcript_update()` at pipeline.py:249:

```
if self._speaking:
    self._was_interrupted = True
    print(f"[Pipeline] Interrupted – buffered: {text[:50]}")
    return          # ← just buffers; does NOT stop audio
```

Then in the TTS pipeline (pipeline.py:627-636):

```
async def start_tts():
    while True:
        sentence = await queue.get()
        if sentence is None or self._was_interrupted:
            await tts_delivery_queue.put(None)  # ← stops NEXT sentence only
```

The self._was_interrupted flag stops the next unstated sentence. The sentence already sent to recall.speak() continues playing. There is no stop_audio() call to Recall.ai — the RecallClient has no such method. The candidate hears the end of the current bot sentence even after they've already started talking again.

What to implement

Fix A — Raise Deepgram endpointing from 300ms → 500ms

File: recall_client.py:51

```
# BEFORE
"endpointing": 300,

# AFTER
"endpointing": 500,
```

Effect: Fewer spurious partial segments mid-sentence. The bot accumulates more of the candidate's answer before the adaptive timer starts. Trade-off: ~200ms slower first-word delivery to pipeline (negligible — silence timer already adds 0.8-1.2s on top).

Fix B — Add stop_audio() to RecallClient and call it on barge-in

Add to recall_client.py:

```
async def stop_audio(self, bot_id: str):
    res = await self._client.post(
        f"{self.base_url}/bot/{bot_id}/output_audio_cancel/",
        timeout=10.0,
    )
    if res.status_code not in (200, 204, 404):
        res.raise_for_status()
```

Add an _on_cancel callback in pipeline.py alongside the existing _on_response callback. Trigger it in on_transcript_update() when self._speaking is True.

Effect: Candidate hears the bot stop within one Recall.ai round-trip (~80-150ms) rather than finishing the full sentence (which could be 2-5 seconds of speech). True barge-in behavior.

Before / After — VAD

Scenario	Before	After
Candidate pauses 400ms mid-sentence (Deepgram fires)	Bot starts 1.2s countdown, may interrupt at 1.5s total	Deepgram holds 500ms, bot waits 1.2s — total 1.7s. Candidate has more time to continue.
Candidate talks over bot	Bot sets flag, current sentence finishes playing (2-5s)	Bot cancels audio via API (~150ms). Candidate takes over immediately.
Very short 3-word burst ("yeah MERN stack")	SILENCE_SHORT + 0.5 = 1.3s then responds	Same — no change needed, already correctly buffered.

Issue 2 — ASR Phonetic Mishearings

What the code actually does today

Deepgram config in recall_client.py:36-54:

```
"deepgram_streaming": {
  "model": "nova-3",
  "language": "multi",
  "smart_format": True,
  "endpointing": 300,
  # NOTE: `keywords` is NOT supported on nova-3 – use `keyterm`
  # instead. However Recall.ai does not document `keyterm` as a
  # passthrough field, so omit it to avoid bot creation failures.
  # nova-3 accuracy is sufficient without keyword boosting.
}
```

The comment at lines 48-51 tells the full story: keyword/keyterm injection was deliberately removed because it was feared to break bot creation. This is the root cause of all the mishearings. With no vocabulary biasing, nova-3 picks the phonetically closest common word.

Confirmed mishearing examples from the June 30 recording:

What candidate said	What Deepgram heard	Bot's response
"MERN stack"	"management development" / "one stick"	"Can you clarify what you mean by one step development?"
"white-label resume"	"right level listen"	Bot hallucinated question about listening
"Gemini and raw PDF"	"DLP (Data Loss Prevention)"	Bot treated it as a DLP topic
"solo project"	"pseudo code"	"Interesting — tell me about the pseudo code"

The system prompt has rule 9 (pipeline.py:101):

```
9. VOICE CALL – STT artifacts: text may have odd spellings or fragments.
   Infer meaning charitably from context. Only flag confusion if meaning is completely lost.
```

But "management development" has a plausible literal meaning, so the LLM does not flag it as garbled — it takes it at face value and builds a wrong mental model.

The eval step has a better disclaimer (pipeline.py:396-401) but it is only in the background evaluation prompt, not in the turn-by-turn interviewer chain.

What to implement

Fix A — Add keyterms to Deepgram config

File: recall_client.py inside the deepgram_streaming dict:

```
"keyterms": [
  "MERN", "React", "Node.js", "Express", "MongoDB", "Next.js",
  "TypeScript", "JavaScript", "Python", "Django", "FastAPI",
  "Supabase", "Firebase", "PostgreSQL", "MySQL", "Redis",
  "Docker", "Kubernetes", "AWS", "GCP", "Azure",
  "white-label", "Gemini", "OpenAI", "LLM", "API",
  "microservices", "REST", "GraphQL", "WebSocket",
],
```

The keyterms array biases the beam-search decoder toward these tokens when acoustically plausible. No server cost, no latency added. Test first — if Recall.ai passes unknown fields to Deepgram, this works immediately.

Fix B — ASR post-processing / transcript cleaner step

Add a `_clean_transcript()` method to `ConversationPipeline` in `pipeline.py`. Call it in `_wait_for_silence()` before `_process_turn`:

```
async def _clean_transcript(self, text: str) -> str:
    resp = await self._openai.chat.completions.create(
        model=self._model, # gpt-4o-mini – fast and cheap
        messages=[{
            "role": "user",
            "content": (
                "This is a real-time speech-to-text transcript from a software developer. "
                "Fix ONLY obvious acoustic errors: mangled tech names, framework names, "
                "acronyms (e.g. 'one stick' → 'MERN', 'management development' → 'MERN stack', "
                "'right level listen' → 'white-label', 'pseudo code' → 'solo project', "
                "'DLP' → 'raw PDF'). Do NOT change anything else. Return only the fixed text.\n\n"
                f"Transcript: {text}"
            ),
        }],
        max_tokens=200,
        temperature=0.1,
    )
    return (resp.choices[0].message.content or text).strip()
```

Change in `_wait_for_silence()`:

```
# BEFORE
await self._process_turn(text, speaker)

# AFTER
cleaned_text = await self._clean_transcript(text)
await self._process_turn(cleaned_text, speaker)
```

Latency cost: ~150-250ms (gpt-4o-mini). This sits inside the existing silence timer buffer and is invisible to the user.

Fix C — Strengthen system prompt rule 9

File: `pipeline.py`:101

```
# BEFORE
9. VOICE CALL – STT artifacts: text may have odd spellings or fragments.
   Infer meaning charitably from context. Only flag confusion if meaning is completely lost.

# AFTER
9. VOICE CALL – STT artifacts: text may have odd spellings or fragments.
   Infer meaning charitably from context. NEVER repeat an unusual or garbled-sounding
   term back to the candidate verbatim. If unsure what was said, use a generic
   follow-up: "Could you tell me a bit more about that?" – never echo the garbled word.
```

Before / After — ASR

What candidate said	Deepgram today	Bot today	Bot after fixes
"MERN stack"	"management development"	"Can you clarify what you mean by one step development?"	"Got it — what did you build with it?"

What candidate said	Deepgram today	Bot today	Bot after fixes
"white-label resume"	"right level listen"	Hallucinates question about listening	"Sure — and what was the end product?"
"solo project"	"pseudo code"	"Interesting — tell me about the pseudo code"	"Right, a solo project — who was the intended user?"

Issue 3 — Contextual Memory & Gaslighting

What the code actually does today

CandidateProfile (pipeline.py:150-173) tracks skills_detected and technologies_detected, populated by the background _evaluate_and_update() task.

The problem chain:

1. Candidate says "MERN stack" → Deepgram hears "management development"
2. on_transcript_update("management development") → silence timer → _process_turn()
3. _process_turn appends {"role": "user", "content": "management development"} to self._history
4. LLM generates a question about "management development" (plausible literal meaning)
5. Background eval adds "management" to skills_detected
6. Candidate corrects: "No, it's MERN stack"
7. self._history still contains "management development" in step 3's turn
8. _build_state_context() at pipeline.py:469 injects a block reinforcing the wrong term
9. This actively reinforces the wrong term into every subsequent turn's context

Step 8 specifically injects:

```
Candidate confirmed skills: management, ...
Technologies mentioned: management development, ...
```

The double-down pattern occurs because:

- Turn N: Bot asks about "management development"
- Candidate corrects: "It's MERN stack"
- Turn N+1: state_ctx still shows skills_detected = ["management"]
- LLM asks "Can you clarify what you mean by one step development?" — now using a third garbled variation from earlier in the history

There is no code-level correction-locking mechanism. Rule 9 in the system prompt is LLM-instruction-only and fails when the history context overrides it.

What to implement

Fix A — Add confirmed_corrections dict to CandidateProfile

```
@dataclass
```

```
class CandidateProfile:
    skills_detected: list = field(default_factory=list)
    technologies_detected: list = field(default_factory=list)
    confirmed_corrections: dict = field(default_factory=dict) # garbled → correct
    # ... rest unchanged
```

Fix B — Correction detection before LLM call

Add `_detect_correction()` to `ConversationPipeline`:

```
def _detect_correction(self, text: str) -> tuple[str, str] | None:
    """Detect patterns like 'not X, it's Y' or 'I mean Y not X'."""
    import re
    patterns = [
        r"(?:it'?s not|not)\s+(.+?)[,.]?\s+it'?s\s+(.+)",
        r"(?:I mean|I said)\s+(.+?)[,.]?\s+not\s+(.+)",
    ]
    for p in patterns:
        m = re.search(p, text, re.IGNORECASE)
        if m:
            return (m.group(1).strip(), m.group(2).strip())
    return None
```

When a correction is detected, update `CandidateProfile.confirmed_corrections` and remove the wrong term from `skills_detected/technologies_detected`.

Fix C — Inject confirmed corrections prominently in `_build_state_context()`

```
if p.confirmed_corrections:
    lines.append(
        "CONFIRMED CORRECTIONS (use these terms ONLY, never the old ones): "
        + ", ".join(
            f"'{old}' = actually '{new}'"
            for old, new in p.confirmed_corrections.items()
        )
    )
```

Fix D — Strengthen system prompt with a correction rule

Add to `_RULES_PREFIX` in `pipeline.py`:

```
8b. CORRECTION RULE – If the candidate explicitly says "it's not X" or "I mean Y":
(a) Immediately acknowledge the corrected term: "Got it, MERN stack."
(b) NEVER use the old garbled term again in this conversation.
(c) Ask your next question using the corrected term only.
```

Before / After — Memory & Gaslighting

Turn	Before	After
T1: Bot hears "management development"	Asks about "management development"	Same (ASR not yet fixed in isolation)
T2: Candidate says "no, MERN stack"	<code>skills_detected</code> still = ["management"]. Bot asks "clarify one step dev?"	Correction detected → <code>confirmed_corrections["management development"]</code> = "MERN stack". Bot says "Got it, MERN stack — what did you build?"

Turn	Before	After
T3: Next topic	State context injects "management" as a skill	State context injects "CONFIRMED: 'management development' = 'MERN stack'" — LLM uses correct term
T10: Scorecard	Scorecard says candidate knows "management"	Scorecard says candidate knows "MERN stack"

Issue 4 — Repetitive Loop Fallbacks

What the code actually does today

System prompt rule 8 at pipeline.py:97-99:

```
8. STT NOISE RULE — ...
   NEVER ask for clarification on the same point twice.
```

There is no code-level enforcement of this rule. It is entirely LLM-instruction-based and the LLM violates it because each garbling sounds like a different unclear thing — so rule 8 "same point" never triggers from the LLM's perspective.

InterviewState.should_advance_phase() at pipeline.py:137 only checks:

- questions_asked >= 3 for intro phase
- elapsed > 40 minutes or topics exhausted for technical
- questions_asked >= 2 for behavioral

There is no confusion-count trigger. The bot can loop on the same misheard topic for 10 turns without ever forcing a pivot.

The only fallback phrase is:

```
"Could you tell me a bit more about that?"
```

When this fires twice in a row, the pattern is immediately obvious to the candidate.

What to implement

Fix A — Add consecutive_confusion_count to InterviewState

```
@dataclass
class InterviewState:
    # ... existing fields
    consecutive_confusion_count: int = 0
    last_topic_at_confusion: str = ""
```

Fix B — Detect clarification responses and count them

After the LLM response is generated in _process_turn, check if it matched a clarification pattern. If yes, increment the counter. Reset to 0 on any substantive (non-clarification) response.

```
CONFUSION_PIVOT_THRESHOLD = 2
```



```
def _is_clarification_response(self, text: str) -> bool:
    patterns = [
        "tell me a bit more", "could you clarify", "can you elaborate", "what do you mean"
    ]
    return any(p in text.lower() for p in patterns)
```

Fix C — Inject forced topic-change directive in `_build_state_context()` at threshold

```
if s.consecutive_confusion_count >= CONFUSION_PIVOT_THRESHOLD:
    lines.append(
        "FORCE TOPIC CHANGE: You have asked for clarification too many times on this point. "
        "Do NOT ask for clarification again. Move immediately to a completely new topic: "
        + (s.topics_remaining[0] if s.topics_remaining else "a behavioral question")
        + ". Acknowledge the candidate briefly and pivot."
    )
```

Fix D — Add a `CONFUSION_FALLBACKS` rotation list

```
CONFUSION_FALLBACKS = [
    "Sure, let's come back to that. Tell me about another project you've worked on recently.",
    "Okay, no worries. Let me ask you something different – what's your experience with system design?",
    "Got it. Let's shift gears – tell me about a technical challenge you've solved recently.",
    "That's fine. Let me ask you about something else – how do you approach debugging a production issue?",
]
```

When a forced pivot fires, select from this list (rotating via index counter) rather than letting the LLM generate the transition. This guarantees variety at the code level.

Before / After — Loop Fallbacks

Turn	Before	After
T1: Garbled answer	Bot: "Could you tell me a bit more about that?"	Same
T2: Still garbled	Bot: "Could you clarify what you mean by one step development?"	confusion_count = 2 → state injects FORCE TOPIC CHANGE directive
T3	Bot: loops again on same garbled term	Bot: "Okay, let's move on — tell me about a technical challenge you solved recently."
T4	Candidate stuck in clarification hell	Candidate gets a fresh question, session continues productively

What's In Code vs What's Missing

Issue	Currently In Code	Missing
VAD silence thresholds	Adaptive timeouts (0.8-3.5s) exist and scale by word count	Deepgram endpointing at 300ms too aggressive; no Recall.ai audio cancel API call

Issue	Currently In Code	Missing
Barge-in	Sets <code>_was_interrupted</code> flag, stops next sentence	Does not cancel audio already sent to Recall.ai
ASR vocabulary	No keyword biasing (deliberately removed in <code>recall_client.py</code> comment)	keyterms passthrough, ASR cleaner LLM step
ASR post-processing	Eval step has charitable-read note (background eval only)	No cleaner step between Deepgram → main interviewer LLM
Correction handling	Rule 9 in system prompt (LLM-only, easily overridden by history context)	<code>confirmed_corrections</code> dict, term locking, code-level correction detection
Confusion loop prevention	Rule 8 in system prompt (LLM-only, fails when each garbling looks different)	<code>consecutive_confusion_count</code> , code-level forced pivot at threshold 2
Fallback phrase variety	<code>BACKCHANNELS</code> array for listening acknowledgments	No <code>CONFUSION_FALLBACKS</code> rotation for error recovery pivots

Priority Order for Implementation

Priority	Fix	File(s)	Effort	Impact
1	Add keyterms to Deepgram config	<code>recall_client.py</code>	5 lines	Directly reduces mishearings at source
2	ASR transcript cleaner step	<code>pipeline.py</code>	~20 lines	Fixes root cause of gaslighting and loops together
3	<code>confirmed_corrections</code> dict + detection	<code>pipeline.py</code>	~30 lines	Eliminates gaslighting once corrections are spoken
4	<code>consecutive_confusion_count</code> + forced pivot	<code>pipeline.py</code>	~20 lines	Breaks the loop fallback pattern
5	Raise Deepgram endpointing 300 → 500ms	<code>recall_client.py</code>	1 line	Reduces mid-sentence interruptions
6	Recall.ai audio cancel on barge-in	<code>recall_client.py</code> + <code>pipeline.py</code>	~25 lines	True barge-in; verify endpoint exists in your Recall.ai region first

Recommended first batch: priorities 1-4. These are pure Python changes with zero risk of breaking the Recall.ai integration. Priorities 5-6 require testing in a live call to validate the timing change and confirm the cancel API endpoint.